
Rapport d'avancement de stage

Semaine 6

Synthese vocale par clonage de voix (XTTS)
pour la generation a la volee de l'audio des pilotes virtuels

Un copilote d'intelligence artificielle pour le controle aerien

Auteur Nicolas Marano
Projet Simulateur de controleur aerien (PoC, 10 semaines)
Etablissement Universite de Reims Champagne-Ardenne (URCA)
Calcul (IA) Supercalculateur ROMEO : NVIDIA GH200 (Grace-Hopper, aarch64)
Domaine Intelligence artificielle & controle du trafic aerien
Periode Semaine 6 du stage

Resume. Cette semaine porte sur le module de synthese vocale, qui ferme la boucle de dialogue en restituant la reponse du pilote en audio. La solution retenue est un clonage de voix *zero-shot* fonde sur le modele XTTS-v2 : a partir d'un court extrait de reference, le systeme genere a la volee une voix distincte pour chaque pilote virtuel, puis applique une degradation VHF (filtre de Butterworth 300-3400 Hz) pour reproduire le canal radio. La synthese est mesuree a 3,9 s par collationnement (facteur temps reel 0,82, CPU) ; le taux d'erreur mots de re-transcription de la voix synthetisee est de 22 a 24 %.

Table des matières

1	Contexte et positionnement de la semaine	1
1.1	Synthese chiffree de la semaine	1
2	Semaine 6 : Synthese vocale par clonage de voix	1
2.1	Methode : clonage <i>zero-shot</i> avec XTTS-v2	1
2.2	Chaine de synthese	2
2.3	Generation du collationnement (phraseologie OACI)	2
2.4	Voix de reference	3
2.5	Degradation VHF et coherence de domaine	3
2.6	Integration : la boucle vocale complete	4
2.7	Resultats et mesures	5
3	Bilan et transition	5

1 Contexte et positionnement de la semaine

Le projet automatise le dialogue radio pilote / controleur a l'interieur d'un simulateur de trafic aerien. Une instruction parlée est transcrite, ancrée dans la phraseologie OACI, transformée en une commande validée, exécutée dans le simulateur BlueSky, puis restituée sous forme d'un *readback* (collationnement) en voix de pilote synthétisée. La chaîne cible se resume ainsi :

voix pilote (VHF) → Whisper STT → NER + LLM avec RAG (OACI Doc 4444) → JSON valide
→ simulateur BlueSky (manoeuvre avion) → voix de readback synthetisee → boucle

Les semaines precedentes ont produit le module de **perception** (semaine 4, reconnaissance vocale par Whisper fine-tune) et le module d'**interpretation** (semaine 5, LLM ancre par RAG produisant un JSON valide et borne par une validation de securite). La semaine 6 traite du **dernier maillon de la boucle : la restitution vocale**. Il s'agit de convertir le texte du collationnement en un signal audio credible, propre a chaque aeronef, afin que le simulateur ne se contente pas d'exécuter la manoeuvre mais reponde egalement sur la frequence comme le ferait un pilote reel.

1.1 Synthese chiffree de la semaine

Indicateur	Resultat
Latence de synthese (par collationnement, CPU)	3,9 s (facteur temps reel 0,82)
WER de re-transcription de la voix synthetisee (S6/S8)	22 a 24 %
WER de reference sur voix humaine reelle (validation S4)	6,7 %
Voix de reference clonees (pilotes / controleur distincts)	3
Bande passante de degradation VHF (Butterworth ordre 6)	300–3400 Hz
Boucle vocale complete rejouee et verifiee	oui (2026-06-11)

Table 1 – Vue d'ensemble des resultats obtenus durant la semaine 6.

2 Semaine 6 : Synthese vocale par clonage de voix

Objectif de la semaine

Implementer une solution de **clonage vocal** permettant de generer a la volee l'audio des differents pilotes virtuels. Chaque aeronef doit pouvoir prononcer son collationnement avec une voix qui lui est propre, sans entrainement specifique par locuteur, et dans un rendu sonore compatible avec le canal radio VHF du simulateur.

2.1 Methode : clonage *zero-shot* avec XTTS-v2

La generation repose sur le modele XTTS-v2 (paquet `coqui-tts`). Le choix d'un clonage *zero-shot* evite tout entrainement par locuteur : le modele reproduit le timbre d'une voix a partir d'un seul extrait de reference de quelques secondes, fourni au moment de la synthese. Cette approche permet d'attribuer une voix distincte a chaque aeronef sans constituer de corpus dedie ni lancer de campagne d'entrainement supplementaire, ce qui est adapte a la contrainte de calcul et de stockage du cluster.

Composant	Valeur	Remarque
Modele	XTTS-v2 (<code>coqui-tts</code>)	multilingue, clonage zero-shot
Strategie	clonage sans entrainement	voix clonee depuis un extrait de reference
Voix de reference	3 extraits ATCO2 (4 a 9 s)	une voix par pilote virtuel
Frequence native	24 kHz → 16 kHz	rééchantillonnage vers l'entree ASR
Langue	en	phraseologie anglaise
Materiel d'inference	CPU (noeud GH200)	contournement du bug cuFFT (GPU)
Post-traitement	degradation VHF 300-3400 Hz	rendu radio (<i>sim-to-real</i>)

Table 2 – Configuration de la synthese vocale (semaine 6).

2.2 Chaine de synthese

La fonction `synth()` enchaîne quatre etapes deterministes apres la generation XTTS : rééchantillonnage de 24 kHz vers 16 kHz (frequence d'entree du pipeline ASR), normalisation d'amplitude pour eviter l'ecretage, puis degradation VHF. La sortie est un signal mono 16 kHz directement consommable par le reste de la chaine.

```
texte de readback (OACI) → XTTS-v2 (clonage, voix de reference) → rééchantillonnage 24→16
                                kHz
                                → normalisation → degradation VHF (Butterworth 300-3400 Hz) → wav 16 kHz
```

Listing 1 – Coeur de la synthese : clonage XTTS puis post-traitement (extrait de `tts_atc.py`).

```
wav = np.asarray(tts.tts(text=text, speaker_wav=ref_wav, language=language), dtype=np.float32)
wav16 = resample_poly(wav, FS, XTTS_SR).astype(np.float32) # 24k -> 16k
peak = np.max(np.abs(wav16)) + 1e-9
if peak > 1.0:
    wav16 = wav16 / peak
if vhf:
    wav16 = preprocess_waveform(wav16, training=False) # bande passante VHF 300-3400 Hz
```

2.3 Generation du collationnement (phraseologie OACI)

Le texte prononce n'est pas l'instruction brute mais un *collationnement* conforme a la phraseologie. Le module `readback.py` convertit les ordres JSON en telephonie standard : chiffres epeles un a un (**niner** pour 9), indicatifs en code telephonique (**AFR** → *air france*), points de cheminement en alphabet OACI. Le sens d'un changement d'altitude (montee ou descente) est determine en comparant la valeur cible a l'altitude courante de l'aeronef.

Listing 2 – Exemple de collationnement genere a partir d'un ordre JSON.

```
Ordre : {callsign: AFR1234, action: ALT, value: 10000} (avion au FL130)
Texte : "descend flight level one zero zero, air france one two three four"
```

2.4 Voix de reference

Les voix de reference sont extraites du corpus ATCO2 (jeu de test) par le module `make_pilot_voices.py` : il selectionne trois segments courts (4 a 9 s, non silencieux) qui servent de timbres a cloner. Chaque aeronef de la flotte se voit attribuer une de ces voix, de sorte que des pilotes differents sonnent differemment sur la frequence. L'usage d'extraits radio reels comme reference contribue au realisme du rendu.

2.5 Degradation VHF et coherence de domaine

La voix synthetisee par XTTS est propre (large bande). Pour la rendre credible sur une frequence aeronautique, la chaine applique la degradation VHF deja validee en semaine 2 : un filtre passe-bande de Butterworth d'ordre 6, limite a la bande 300–3400 Hz (figure 1). Cette etape a un second interet, au-dela du realisme : elle place la voix synthetisee dans le *memes domaine acoustique* que les donnees sur lesquelles le module de reconnaissance vocale a ete specialise. La re-transcription du collationnement reste ainsi coherente avec le reste de la chaine (reduction du fossé *sim-to-real*). La figure 2 illustre l'effet de ce filtrage sur un signal de parole.

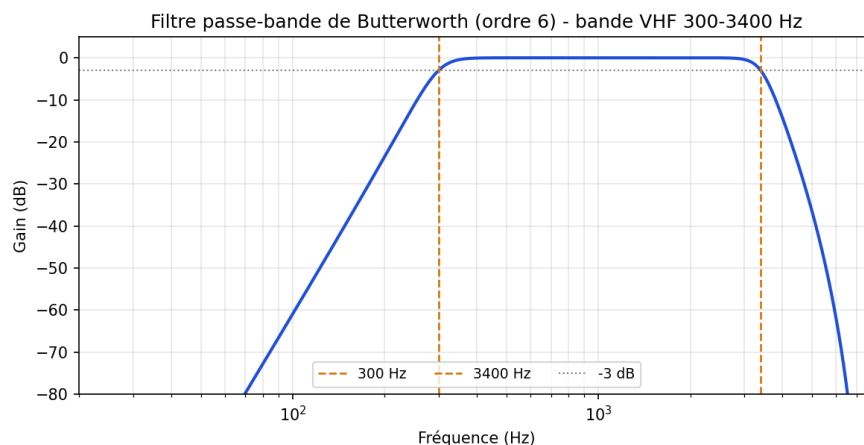


Figure 1 – Reponse en frequence du filtre passe-bande de Butterworth (ordre 6) utilise pour la degradation VHF. La bande utile est limitee a 300–3400 Hz, comme une radio aeronautique.

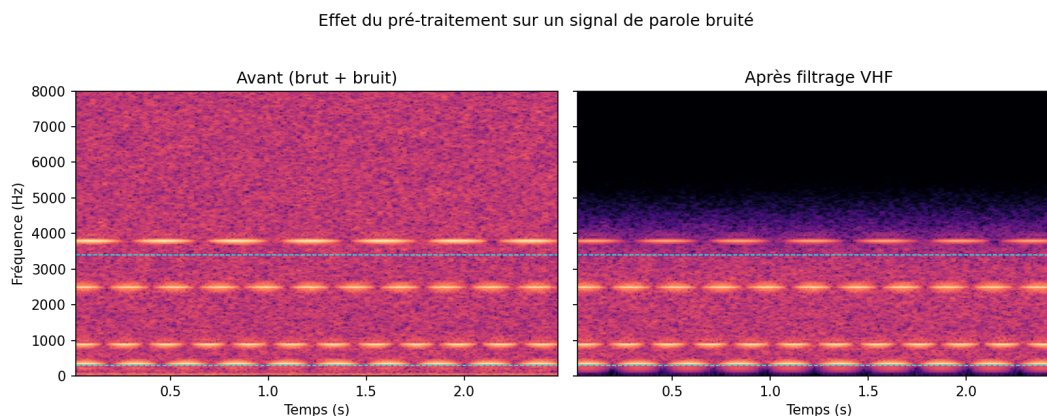


Figure 2 – Effet de la dégradation VHF sur un signal de parole : spectre avant (gauche) et après filtrage passe-bande (droite). Le contenu hors bande 300–3400 Hz est atténué, reproduisant la coloration d’un canal radio. La même chaîne est appliquée à la voix synthétisée.

Contraintes techniques et choix d’ingénierie

L’intégration de XTTS dans l’environnement logiciel du cluster a nécessité plusieurs ajustements. Le paquet `coqui-tts` a été ajouté à l’environnement existant (*whisper-attc*) plutôt que dans un environnement séparé, afin de réutiliser la même version de PyTorch et de `transformers` : ce choix évite une seconde installation de PyTorch, tient dans le quota disque de 20 Go et permet de servir les trois modèles (Whisper, Mistral, XTTS) depuis un unique processus. Comme XTTS importe un symbole (`isin_mps_friendly`) supprimé dans `transformers` 5.x, ce symbole est restauré par un correctif appliqué avant l’import de la bibliothèque, sans retrograder `transformers` (qui doit rester en version 5.9 pour Whisper et Mistral). Enfin, l’encodeur de locuteur de XTTS déclenche un bug `cuFFT` de `torchaudio` sur le GPU GH200 ; la synthèse est donc exécutée sur CPU, ce qui la rend fiable mais non temps réel.

2.6 Intégration : la boucle vocale complète

L’assemblage est valide par le module `voice_exchange.py`, qui rejoue un échange radio complet. Pour chaque instruction, le contrôleur parle (synthèse + VHF), l’audio est transcrit puis interprété, BlueSky exécute l’ordre, et l’aéronef collationne dans sa voix clonée (synthèse + VHF). Le collationnement est ensuite re-transcrit pour vérifier son intelligibilité, et les signaux sont assemblés en fichiers d’échange et en une bande son de session. La boucle complète *voix* → *Whisper* → *Mistral+RAG* → *BlueSky* → *readback vocal* a été jouée et vérifiée le 2026-06-11 (descente FL280 → FL180 exécutée puis collationnée), comme l’illustre la figure 3.

2.7 Resultats et mesures

Resultats cles

- **Latence de synthèse** : 3,9 s par collationnement, facteur temps réel 0,82 (XTTS sur CPU, $n = 3$).
- **WER de re-transcription** de la voix synthétisée : 22 à 24 % (mesure sur la boucle S6/S8).
- **Boucle vocale complète** jouée et vérifiée de bout en bout (2026-06-11).
- **Trois voix de référence** clonées, attribuées à des aéronefs distincts.



Figure 3 – Application en mode ROMEO : apres une clairance vocale, l’aeronef AFR300 est etabli au FL180 et collationne dans sa voix clonee. La transcription de l’instruction apparait sous le bouton *push-to-talk*.

Analyse. La re-transcription de la voix synthetisee atteint un WER de 22 a 24 %, contre 6,7 % pour une voix humaine reelle (validation de la semaine 4). Cet ecart est attendu : la boucle voix-clonee → Whisper cumule les erreurs de deux modeles successifs (la synthese et la reconnaissance), et la voix synthetique presente des caracteristiques hors du domaine d’entrainement du module de reconnaissance. Ce taux confirme neanmoins que le collationnement reste *intelligible* et exploitable pour un retour radio dans un contexte d’entrainement. La synthese reste par ailleurs stochastique : sur un meme texte, une generation propre et une generation plus degradee ont ete observees. Avec un facteur temps reel de l’ordre de 0,82 sur CPU, la synthese n’autorise pas un flux continu en temps reel, mais reste compatible avec le rythme d’une frequence reelle (un echange toutes les dizaines de secondes).

3 Bilan et transition

La semaine 6 complete la boucle de dialogue par un module de restitution vocale. Le clonage *zero-shot* XTTS-v2 genere a la volee une voix distincte par aeronef sans entrainement par locuteur, et la degradation VHF reutilisee de la semaine 2 assure a la fois le realisme radio et la coherence de domaine avec le module de reconnaissance. L’integration a ete validee de bout en bout : une clairance parlee provoque la manoeuvre de l’aeronef dans BlueSky, suivie d’un collationnement audible dans la voix clonee du pilote.

Prochaines etapes. Les etapes suivantes visent a alimenter le LLM par l’etat de trafic courant (requete situee), puis a consolider la mise en service temps reel de la boucle complete et a mener une evaluation quantitative sur des scenarios complets. Les limites identifiees cette semaine (variance de la synthese, execution CPU non temps reel, WER de re-transcription accru) orientent ces travaux, notamment vers une attenuation des cas de synthese degradee et une mesure systematique de l’intelligibilite.